# 13. Construct a C Program to Implement Various Memory Allocation Strategies

**Code - Part 1 (Lines 1–35) — First Fit**

```c
1  #include <stdio.h>
2
3  void firstFit(int mp[], int p[], int m, int n)
4  {
5      int alloc[m];
6
7      for(int i = 0; i < m; i++)
8          alloc[i] = -1;
9
10     for(int i = 0; i < m; i++)
11     {
12         for(int j = 0; j < n; j++)
13         {
14             if(mp[j] >= p[i])
15             {
16                 alloc[i] = j;
17                 mp[j] -= p[i];
18                 break;
19             }
20         }
21     }
22
23     printf("\nProcess\tSize\tBlock\n");
24     for(int i = 0; i < m; i++)
25     {
26         if(alloc[i] != -1)
27             printf("P%d\t%d\t%d\n", i+1, p[i], alloc[i]+1);
28         else
29             printf("P%d\t%d\tNot Allocated\n", i+1, p[i]);
30     }
31 }
32
33 void bestFit(int mp[], int p[], int m, int n)
34 {
35     int alloc[m];
```

**Code - Part 2 (Lines 36–70) — Best Fit**

```
36
37      for(int i = 0; i < m; i++)
38          alloc[i] = -1;
39
40      for(int i = 0; i < m; i++)
41      {
42          int best = -1;
43          for(int j = 0; j < n; j++)
44          {
45              if(mp[j] >= p[i])
46              {
47                  if(best == -1 || mp[j] < mp[best])
48                      best = j;
49              }
50          }
51
52          if(best != -1)
53          {
54              alloc[i] = best;
55              mp[best] -= p[i];
56          }
57      }
58
59      printf("\nProcess\tSize\tBlock\n");
60      for(int i = 0; i < m; i++)
61      {
62          if(alloc[i] != -1)
63              printf("P%d\t%d\t%d\n", i+1, p[i], alloc[i]+1);
64          else
65              printf("P%d\t%d\tNot Allocated\n", i+1, p[i]);
66      }
67  }
68
69  void worstFit(int mp[], int p[], int m, int n)
70  {
```

**Code - Part 3 (Lines 71–103) — Worst Fit**

```
71      int alloc[m];
72
73      for(int i = 0; i < m; i++)
74          alloc[i] = -1;
75
76      for(int i = 0; i < m; i++)
77      {
78          int worst = -1;
79          for(int j = 0; j < n; j++)
80          {
81              if(mp[j] >= p[i])
82              {
83                  if(worst == -1 || mp[j] > mp[worst])
84                      worst = j;
85              }
86          }
87
88          if(worst != -1)
89          {
90              alloc[i] = worst;
91              mp[worst] -= p[i];
92          }
93      }
94
95      printf("\nProcess\tSize\tBlock\n");
96      for(int i = 0; i < m; i++)
97      {
98          if(alloc[i] != -1)
99              printf("P%d\t%d\t%d\n", i+1, p[i], alloc[i]+1);
100         else
101             printf("P%d\t%d\tNot Allocated\n", i+1, p[i]);
102     }
103 }
104
```

**Code - Part 4 (Lines 108–142) — Main Function**

```
108
109     printf("Number of memory partitions: ");
110     scanf("%d", &n);
111
112     printf("Number of processes: ");
113     scanf("%d", &m);
114
115     printf("Enter memory partitions:\n");
116     for(int i = 0; i < n; i++)
117         scanf("%d", &mp[i]);
118
119     printf("Enter process sizes:\n");
120     for(int i = 0; i < m; i++)
121         scanf("%d", &p[i]);
122
123     printf("\n1. First Fit\n2. Best Fit\n3. Worst Fit\nEnter choice: ");
124     scanf("%d", &ch);
125
126     switch(ch)
127     {
128         case 1:
129             firstFit(mp, p, m, n);
130             break;
131         case 2:
132             bestFit(mp, p, m, n);
133             break;
134         case 3:
135             worstFit(mp, p, m, n);
136             break;
137         default:
138             printf("Invalid choice\n");
139     }
140
141     return 0;
142 }
```

**Output**

```
Number of memory partitions: 5
Number of processes: 4
Enter memory partitions:
100 500 300 200 400
Enter process sizes:
212 321 211 214

1. First Fit
2. Best Fit
3. Worst Fit
Enter choice: 1

Process Size    Block
P1      212     2
P2      321     5
P3      211     2
P4      214     3


...Program finished with exit code 0
Press ENTER to exit console.
```